

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Τεχνολογίες Υπηρεσιών Λογισμικού

Τεχνολογίες Υπηρεσιών Λογισμικού

Β.Βεσκούκης, Καθ.ΕΜΠ (v.vescoukis@cs.ntua.gr)

Message Queues Lab: RabbitMQ

Χρίστος Χατζηχριστοφή, υ.δ.ΕΜΠ

Introduction to Messaging

- It is the process of sending and receiving data or information between applications
- It is widely used in software development for building distributed applications
- Allows applications to communicate with each other in a decoupled and asynchronous way
- Several advantages (tbd)

Use Cases for Messaging in Software Development

- Send data between applications
- Triggering actions
- Event-driven architecture
- Microservices communication

Advantages of Messaging

- Async communication between applications: sender and receiver do not need to be available at the same time
- Decoupling: Apps can be developed and deployed independently of each other
- Scalability: Allows messages to be distributed accros multiple instances or nodes
- Fault tolerance: Allows messages to be retried or routed to backup applications in case of errors

Comparison of Messaging with Other Communication Methods

- HTTP & direct API calls are synchronous communication methods while messaging is asynchronous
- HTTP & direct API calls are typically point-to-point communication methods (tightly coupled) while messaging allows for loose coupling between apps
- Messaging offers advanced features i.e. guaranteed delivery, message persistence, pub/sub models etc

Messaging systems Overview

- RabbitMQ: open-source broker that implements Advanced Messaging Queuing Protocol (AMQP)
  - provides a wide range of features such as message persistence, message routing, and message priority
- Apache Kafka: distributed streaming platform which allows processing large amounts of data in real-time
  - provides features such as scalability, fault tolerance, and data retention
- ActiveMQ: open-source message broker that supports multiple messaging protocols such as AMQP, MQTT, and STOMP
  - provides features such as message grouping, message filtering, and message expiration

Choosing the right Messaging System

Consider the following

- Use case
- Scalability
- Reliability
- Budget
- Integration
- Maintenance

RabbitMQ Agenda

- AMPQ Overview
- AMPQ Key Concepts
- RabbitMQ Overview
- RabbitMQ Message Flow
- Exchange Types
- How to Install
- Demo

AMPQ Overview (1)

- Advanced Message Queuing Protocol enables conforming client apps to communicate with conforming messaging middleware brokers
- Messaging brokers receive messages from publisher and route them to consumers

AMPQ Overview (2)

- Since it is a network protocol the publishers, consumers and broker can all reside on different machines
- A producer is a user application that sends messages
- A queue is a buffer that stores messages
- A consumer is a user application that receives messages

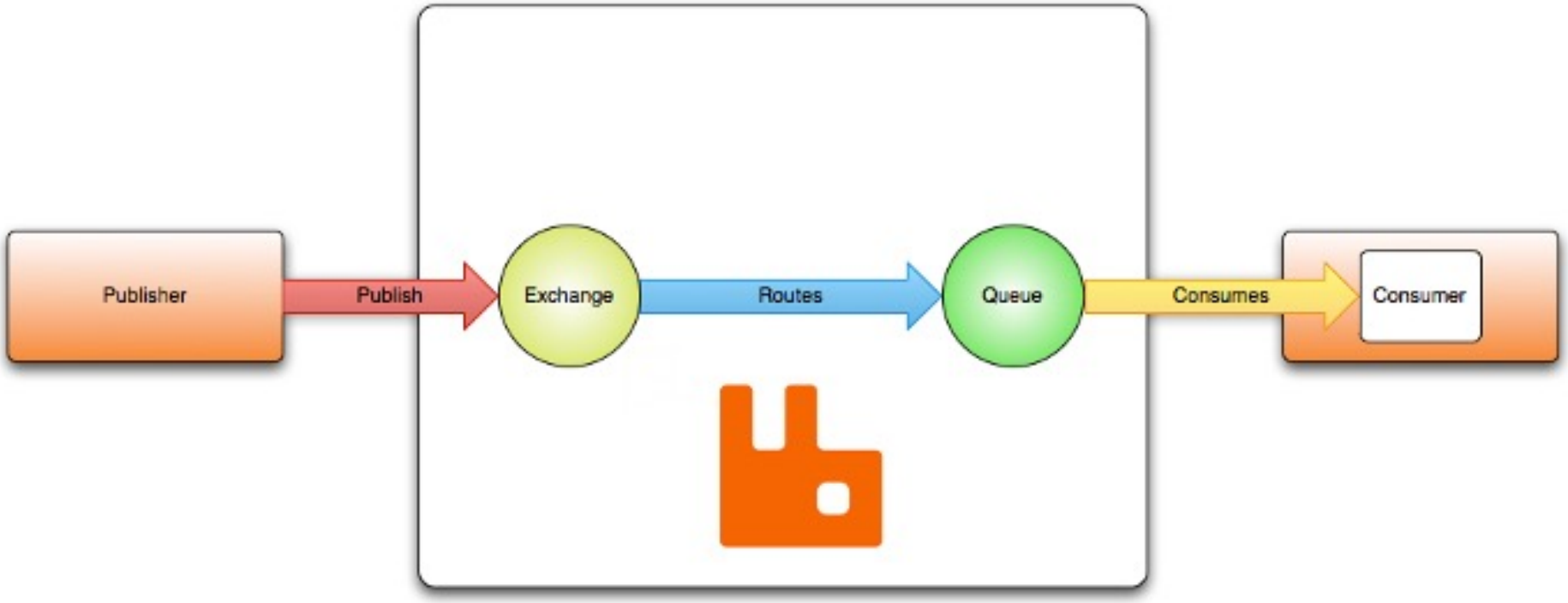
AMPQ Key Concepts (1)

- **Connection:** A connection is a TCP connection between your app and the RabbitMQ broker. AMPQ is an application-level protocol that uses TCP for reliable delivery
- **Channel:** Virtual connection inside a connection. When publishing or consuming messages from a queue – all done over a channel

AMPQ Key Concepts (2)

- **Routing Key:** Routing key binding with the queue are rules that allow the exchange to put messages into the queue
- **Exchange:** The exchange receives messages from the producer and from the other side it pushes them to queues
- **Binding:** A binding is a “link” that you set up to bind a queue to an exchange

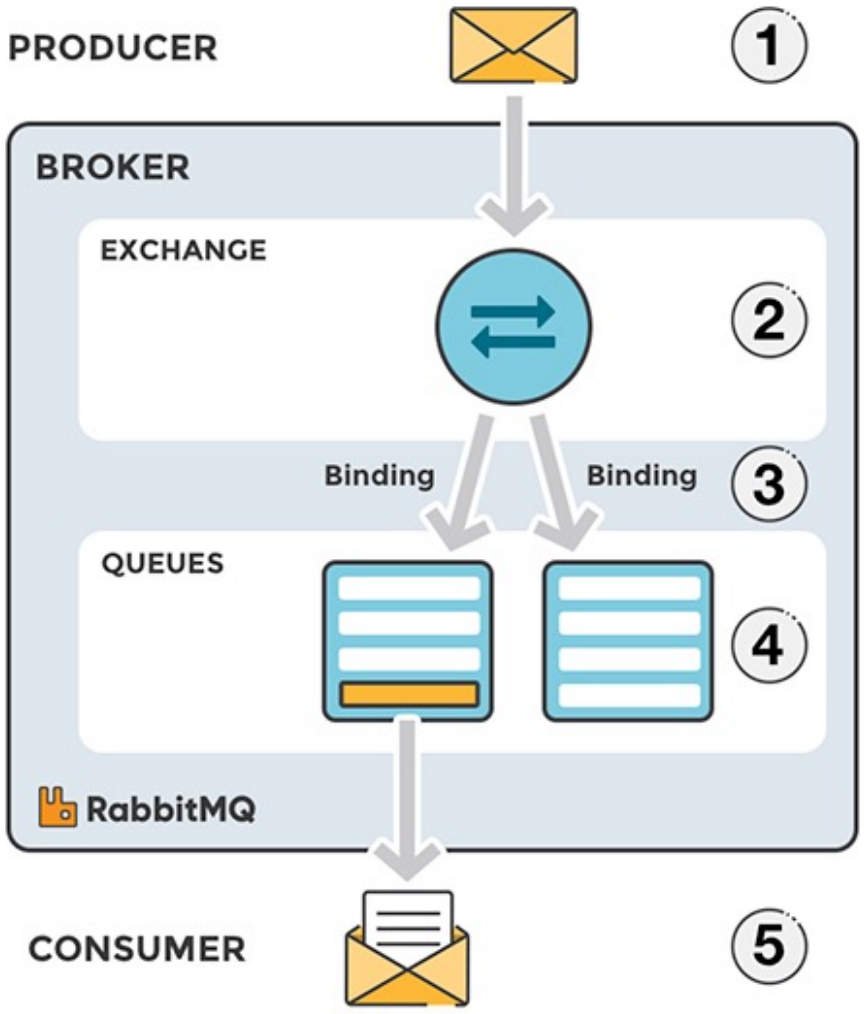
RabbitMQ Overview



- Open-source message broker written in Erlang
- Implements the AMPQ protocol

RabbitMQ Message Flow

- Producer publishes message to the exchange
- Exchange receives the message and is now responsible for the routing of the message
- A binding has to be set up between the queue and the exchange
- The messages stay in the queue until they are handled by a consumer
- The consumer consumes and handles the message



Exchange Types

- Direct exchange
- Fanout exchange
- Topic exchange
- Headers exchange

- Each type of exchange has its own routing rules, which can be used to customize how messages are sent to queues. By using the appropriate exchange type and routing rules, it is possible to create complex routing topologies that can handle a wide variety of messaging scenarios.

Direct Exchange

- Delivers messages to queues based on a message routing key
- Routing key is a message attribute added to the message header by the producer
- Routing key is used by the exchange to decide how to route the message to the appropriate queue(s)
- Message is routed to the queue(s) with the binding key that exactly matches the routing key of the message
- Useful for distinguishing messages published to the same exchange using a simple string identifier

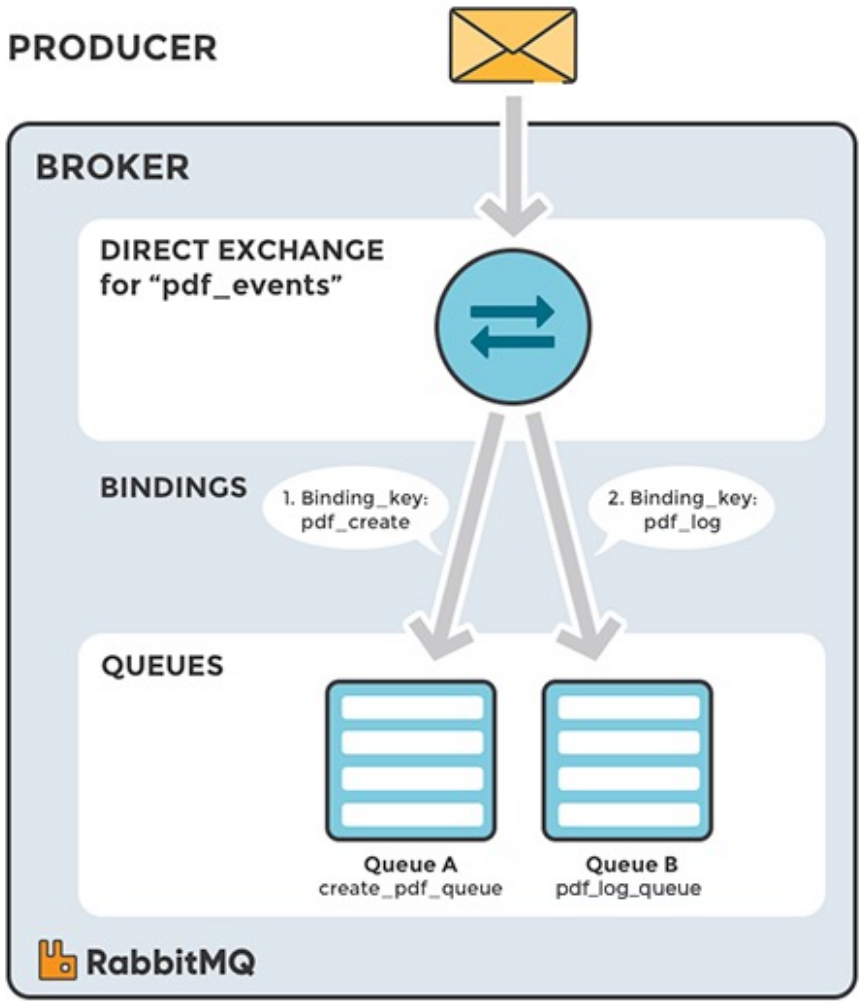
Direct Exchange - Example

SCENARIO 1

- Exchange: pdf_events
- Queue A: create_pdf_queue
- Binding key between exchange (pdf_events) and Queue A (create_pdf_queue): pdf_create

SCENARIO 2

- Exchange: pdf_events
- Queue B: pdf_log_queue
- Binding key between exchange (pdf_events) and Queue B (pdf_log_queue): pdf_log



Topic Exchange

- Routes messages to queues based on wildcard matches between routing key and routing pattern specified by the queue binding
- Messages are routed to one or many queues based on matching between message routing key and pattern
- Routing key must be a list of words delimited by a period (.)
- Routing patterns can contain an asterisk (*) to match a word in a specific position of routing key and a pound symbol (#) to indicate a match of zero or more words
- Consumers indicate their interest in specific topics by setting up a queue and binding it with a given routing pattern to the exchange
- All messages with routing key that match the routing pattern are routed to the queue and stay there until the consumer consumes the message

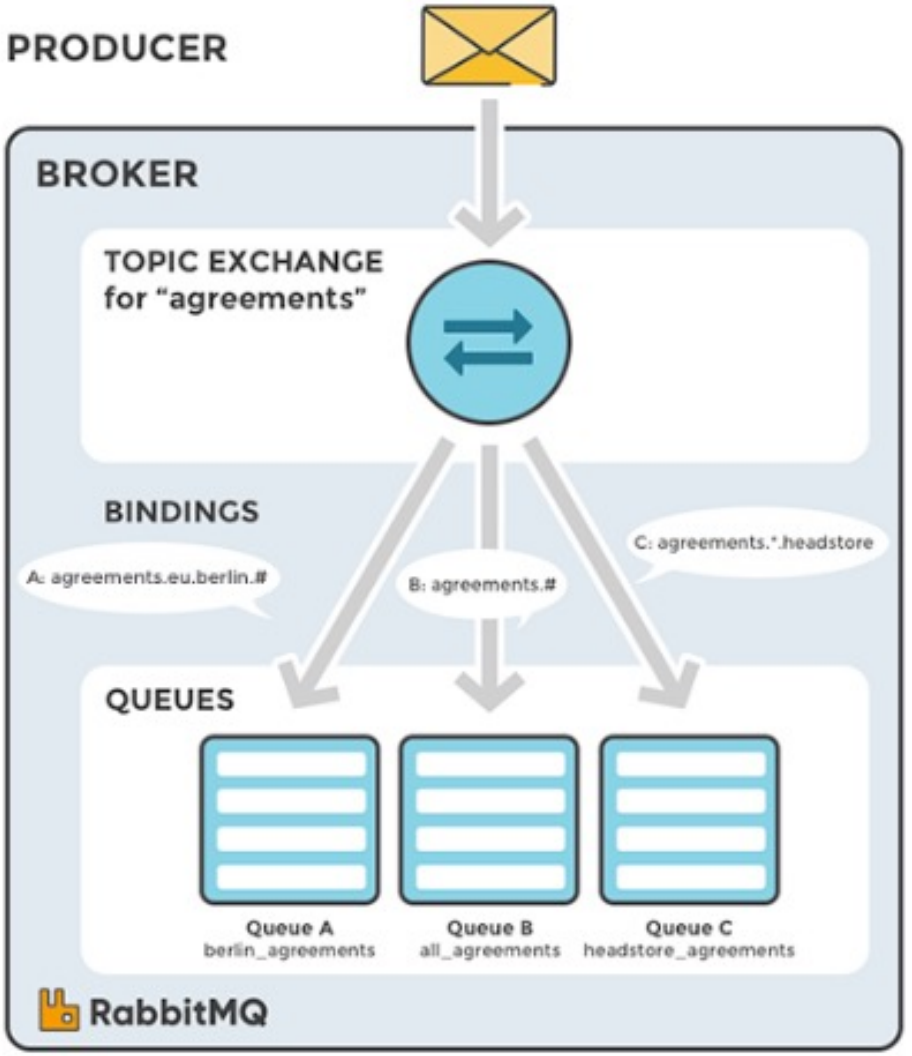
Topic Exchange - Example

SCENARIO 1

- Exchange: agreements
- Queue A: berlin_agreements
- Routing pattern between exchange (agreements) and Queue A (berlin_agreements): agreements.eu.berlin.#
- Routing key that matches example: agreements.eu.berlin, agreements.eu.berlin.headstore

SCENARIO 2

- Exchange: agreements
- Queue B: all_agreements
- Routing pattern between exchange (agreements) and Queue B (all_agreements): agreements.#
- Routing key that matches example: agreements.eu.berlin, agreements.us



Fanout Exchange

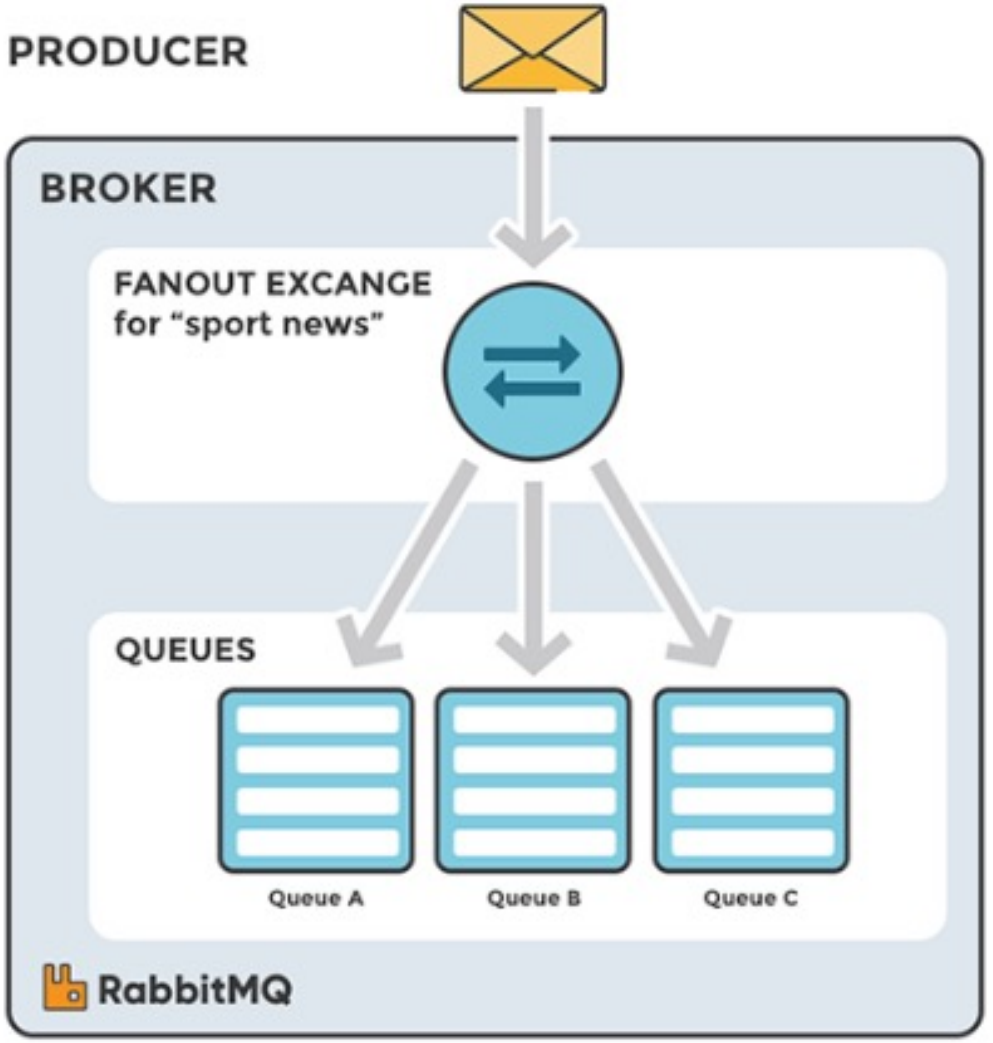
- Copies and routes a message to all queues that are bound to it, regardless of routing keys or pattern matching
- Ignores routing keys provided
- Useful when the same message needs to be sent to one or more queues
- Suitable for scenarios where different consumers need to process the same message in different ways

Fanout Exchange - Example

SCENARIO 1

- Exchange: sport_news
- Queue A: Queue A
- Binding: Binding between the exchange (sport_news) and Queue A

The message will be broadcasted to **every** queue regardless the routing key



Headers Exchange

- Routes messages based on arguments containing headers and optional values
- Similar to topic exchanges, it routes messages based on specific header values rather than routing keys
- A message is considered a match if the value of its header equals the value specified upon binding
- The "x-match" property, added in the binding between exchange and queue, specifies if all headers must match or just one
- "x-match" can have two values: "any" (at least one header pair must match) or "all" (all header pairs must match)
- Headers can be constructed using a wider range of data types, including integer and hash
- The headers exchange type is useful for directing messages containing a subset of known (unordered) criteria

Headers Exchange - Example

SCENARIO 1

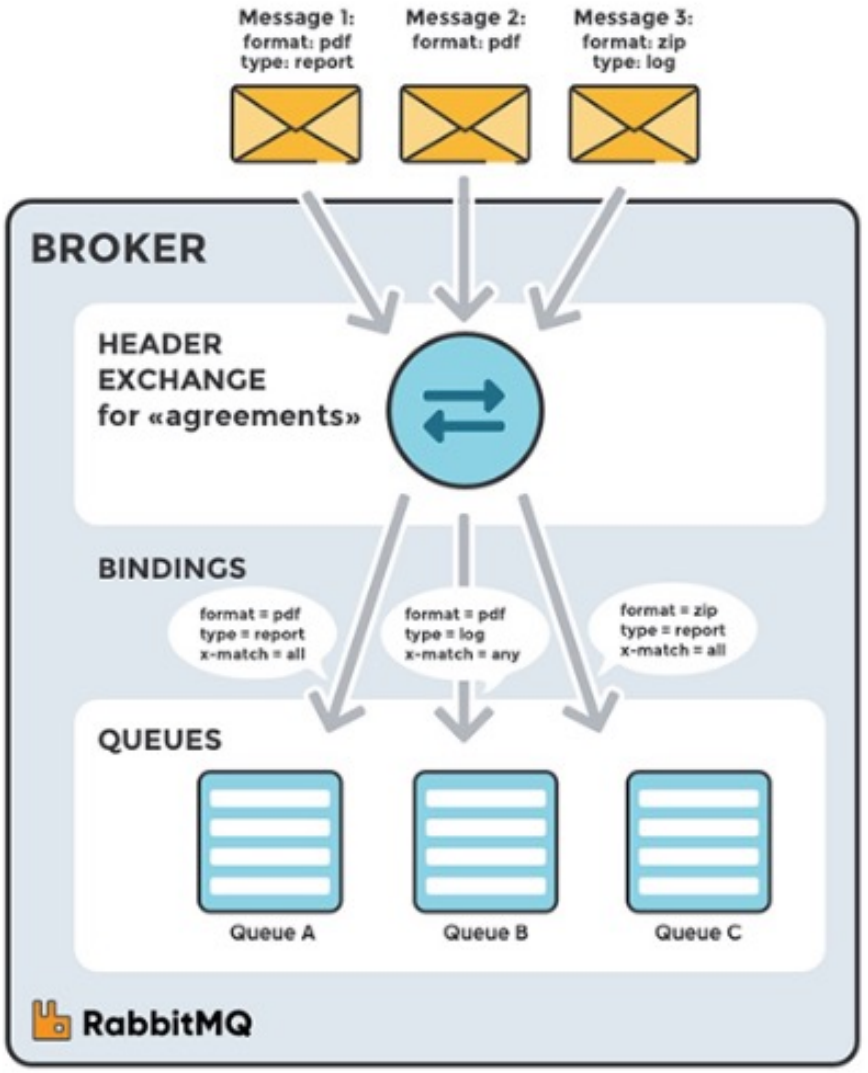
- Exchange: agreements
- Queue A: Queue A
- Binding: rule = ”x-match=any” of format = pdf, type = report

SCENARIO 2

- Exchange: agreements
- Queue B: Queue B
- Binding: rule = ”x-match=any” of format = pdf, type = log

SCENARIO 3

- Exchange: agreements
- Queue C: Queue C
- Binding: rule = ”x-match=all” of format = zip, type = report



Installation Guide - Docker

- If Docker is not installed on your machine:
  - Download from the official docs

- Install RabbitMQ (docker):
  - docker run -d --hostname <hostname> --name <name> -p 5672:5672 -p 15672:15672 rabbitmq:3-management

- Visit localhost:15672
  - Username: guest (default)
  - Password: guest (default)

HANDS-ON

- Produce and Consume messages (NodeJS)
  - Exchange: Direct
  - Number of consumers: 2
  - Number of producers: 2

Useful Links

- RabbitMQ
- RabbitMQ Exchanges, routing keys, bindings
- RabbitMQ Demo – GitHub Repository

Thanks for your attention!

